MATLAB ne travaille qu'avec les matrices, au sens large du terme. Ainsi, toute variable est une matrice. Un scalaire (réel ou complexe) est une matrice de dimension $1 \times 1$. De même une chaîne de caractères est une matrice de chaînes de caractères de dimension $1 \times 1$. En plus des opérations usuelles sur les matrices, MATLAB dispose d'opérateurs qui agissent élément par élément, permettant de se passer souvent des boucles.

## 1 Création d'un vecteur ou matrice

Un vecteur ou matrice peut être défini explicitement en énumérant ses éléments entre crochets [ ]. Les lignes sont séparées par des ; (ou retour-chariot) et les élements d'une même ligne sont séparés par des virgules ou des blancs. Enfin, [ ] désigne la matrice vide.

```
>> a=[1 2 3 ; 4 5 6 ], b=[1+2*i; 3; i], c=[]
a  =

1.   2.   3.
4.   5.   6.

b  =

1. + 2.i
3.
i

c  =

[]
```

On peut utiliser l'opérateur : (énumération) pour créer un vecteur. La syntaxe générale est `v=debut:pas:fin`. Par défaut, le pas est 1.

```
>> v=-1:.5:3
v  =

-1.  -0.5   0.   0.5   1.   1.5   2.   2.5   3.

>> u=-1:3
u  =

-1.   0.   1.   2.   3.

>> w=[-6 -5 -1:3]
w  =

-6.  -5.  -1.   0.   1.   2.   3.
```

Les commandes `linspace` et `logspace` permettent de créer des suites de nombres équidistants entre deux bornes:

- `linspace(a,b,n)` génère un vecteur ligne de $n$ composantes allant exactement de $a$ à $b$ à pas constant. (en l'absence de l'argument $n$, le vecteur généré est de taille 100)

- `logspace(a,b,n)` définit un vecteur dont la première composante est $10^a$, la dernière $10^b$ et dont les composantes sont logarithmiquement équidistantes (i.e leurs logarithmes sont équidistants)

```
>> linspace(.25,1,5)
ans  =

0.25   0.438   0.625   0.813   1.

>> logspace(1,2,5)
ans  =

10.   17.78   31.62   56.23   100.
```

La transposée d'une matrice se fait grâce à l'apostrosphe.

```
>> a'
ans  =

1.   4.
2.   5.
3.   6.
```

- On peut accéder à un élément d'une matrice : il suffit de préciser son numéro de ligne et celui de colonne entre parenthèses
- la fonction `length` renvoie la taille d'un vecteur
- la fonction `size` renvoie la "taille" d'une matrice : nombre de lignes et nombre de colonnes

```
>> b(1,1)
ans  =

1. + 2.i

>> size(a)
ans  =
2.   3.

> size(a,1)
ans  =
2.

> size(a,2)
ans  =
3.
```

donne le nb de lignes

donne le nb de colonnes

```
>> c
c =
[]
```

```
>> size(c)

ans =
0     0
```

Si $a$ est une matrice de dimension $n \times m$, l'instruction `a(i,j)=s` où $i > n$ et $j > m$ crée une matrice de dimension $\max(i, n) \times \max(j, m)$ obtenue en complétant la matrice $a$ avec des 0, et avec $a(i, j) = s$.

```
>> d=[2 2 ; 1 1]
d  =

2.   2.
1.   1.

>> d(3,3)=9
d  =

2.   2.   0.
1.   1.   0.
0.   0.   9.

>> c(3,3)=1 % fonctionne aussi si c n'est pas créé au préalable
c  =

0.   0.   0.
0.   0.   0.
0.   0.   1.
```

## 2 Quelques matrices prédéfinies

- Matrice identité avec la commande `eye`

```
--> eye(2,2)
ans  =

1.   0.
0.   1.

-> eye(2,3)
ans  =

1.   0.   0.
0.   1.   0.
```

- Matrice nulle avec la commande `zeros`

```
--> zeros(3,2)
ans  =
```

```
0.    0.
0.    0.
0.    0.
```

- Matrice ne contenant que des 1 avec la commande `ones`

```
--> ones(2,3)
ans  =

1.    1.    1.
1.    1.    1.
```

- Matrice aléatoire (générée avec une loi uniforme dans $[0,1]$) avec la commande `rand`, et pour une matrice générée de manière aléatoire selon une loi normale avec la commande `randn`

```
>> rand(2,2)
ans  =

0.0975    0.5469
0.2785    0.9575

>> randn(2,2)
ans  =

 2.7694    3.0349
-1.3499    0.7254
```

- Matrice de Hilbert

  C'est la matrice carrée $H = (h_{ij})$ de dimension $n \times n$ définie par :

$$h_{ij} = \frac{1}{i+j-1}, i, j = 1, \ldots, n$$

```
>> hilb(4)

ans =
1.0000    0.5000    0.3333    0.2500
0.5000    0.3333    0.2500    0.2000
0.3333    0.2500    0.2000    0.1667
0.2500    0.2000    0.1667    0.1429
```

## 3 Opérations sur les matrices

Soient $a$ et $b$ deux matrices de mêmes dimensions $n \times m$. Les instructions `a+b` et "`a-b` ont leur sens mathématique usuel. Les instructions `a.*b`, `a./b` et `a.^b` signifient que l'opération s'effectue **élément par élément** :

| | | |
|---|---|---|
| `c=a.*b` | signifie que | $c_{ij} = a_{ij} * b_{ij}$ |
| `c=a./b` | signifie que | $d_{ij} = a_{ij}/b_{ij}$ si $b_{ij} \neq 0$ |
| `e=a.^b` | signifie que | $e_{ij} = a_{ij}^{b_{ij}}$ |

Si $a$ et $b$ sont deux matrices de dimensions $n \times m$ et $m \times q$ alors l'instruction `a * b` a son sens mathématique usuel.
L'instruction `a+b` où $a$ est un scalaire et $b$ une matrice signifie `a*ones(n,m)+b`
Fonctions mathématiques prédéfinies sur les matrices :

on peut additionner un scalaire à une matrice

| | |
|---|---|
| `sin(x)` | |
| `cos(x)` | |
| `tan(x)` | |
| `asin(x)` | |
| `acos(x)` | |
| `atan(x)` | |
| `sqrt(x)` | $\sqrt{x}$ |
| `abs(x)` | $\lvert x \rvert$ |
| `exp(x)` | $e^x$ |
| `log(x)` | $\ln x$ |
| `log10(x)` | logartithme à base 10 |
| `x^y` | $x^y$ |
| `floor(x)` | partie entière |
| `sign(x)` | signe |

```
>> a=ones(2,2)
a  =
1.   1.
1.   1.

>> b=rand(2,2)
b  =
 0.0975    0.5469
0.2785    0.9575


>> a+b
ans  =
1.0975    1.5469
1.2785    1.9575

>> a*b
ans  =
0.3760    1.5044
0.3760    1.5044

>> a.*b
ans  =
0.0975    0.5469
0.2785    0.9575

>> log(a)
ans  =
0.   0.
0.   0.
```

```
>> b.^a

ans =
0.0975    0.5469
0.2785    0.9575

>> cos(a)

ans =
0.5403    0.5403
0.5403    0.5403
```

# 4 Manipuler les vecteurs et matrices

## 4.1 Conversion matrice-vecteur

Une matrice peut être transformée en vecteur grâce à la commande :
Le vecteur est alors composé des colonnes de la matrice mises bout à bout.

```
>> b

b =
0.0975    0.5469
0.2785    0.9575

>> u=b(:)

u =
0.0975
0.2785
0.5469
0.9575
```

Pour convertir un vecteur en matrice on utilise la fonction `reshape` :
`reshape(u,n,m)` découpe le vecteur $u$ en $n$ vecteurs de longueur $m$. Le vecteur $u$ doit être de la bonne taille, à savoir $nm$.

```
>> u=linspace(0,1,6)

u =
0    0.2000    0.4000    0.6000    0.8000    1.0000

>> reshape(u,3,2)

ans =
0    0.6000
0.2000    0.8000
0.4000    1.0000
```

### 4.2 Cas de matrices diagonales

La fonction `diag` permet de créer des matrices diagonales ou d'extraire la diagonale d'une matrice.

- extraction de diagonale : la commande `diag(a)` rend un vecteur contenant les éléments diagonaux de la matrice $a$, même si $a$ n'est pas carrée
- construction d'une matrice à partir d'une diagonale : la commande `diag(u)` crée une matrice carrée dont la diagonale est le vecteur (ligne ou colonne) $u$ et les autres éléments nuls,
- la commande `diag(u,i)` pour $i > 0$ (resp $i < 0$) rend une matrice carrée dont la sur-diagonale numéro $i$ (resp sous-diagonale) est $u$ et les autres éléments nuls.

```
>> a=[1 2 3 ; 4 5 6 ]

a =
1     2     3
4     5     6

>> diag(a)
ans =
1
5

>> diag([1 3 ])
ans =
1     0
0     3

>> diag(a,1)
ans =
2
6
```

### 4.3 Cas des matrices triangulaires

La fonction `triu` (upper) donne la partie triangulaire supérieure d'une matrice et `tril` (lower) donne la partie triangulaire inférieure.

```
>> a
a =
1     2     3
4     5     6

>> tril(a)

ans =
1     0     0
4     5     0

>> triu(a)
```

```
ans =
1     2     3
0     5     6
```

L'inverse d'une matrice inversible est donné par `inv` et le déterminant par `det`

### *4.4 Cas de matrices par blocs*

Particulièrement facile en MATLAB : on veut par exemple construire la matrice par blocs $B = \begin{pmatrix} A & I_3 \\ 0 & A \end{pmatrix}$ où $A$ est la matrice $A = \begin{pmatrix} 3 & 3 & 3 \\ 3 & 3 & 3 \\ 3 & 3 & 3 \end{pmatrix}$

```
>> A=3*ones(3,3);

>> I3=eye(3,3);

>> zero3=zeros(3,3);

>> B=[A I3; zero3, A]
```

on peut ne pas mettre la virgule

```
B  =
3.   3.   3.   1.   0.   0.
3.   3.   3.   0.   1.   0.
3.   3.   3.   0.   0.   1.
0.   0.   0.   3.   3.   3.
0.   0.   0.   3.   3.   3.
0.   0.   0.   3.   3.   3.
```

On peut repérer un élément dans une matrice grâce à un seul indice :

```
>> M=rand(4,4)
M  =

0.308   0.362   0.332   0.269
0.933   0.292   0.594   0.633
0.215   0.566   0.502   0.405
0.313   0.483   0.437   0.918

>> M(16)
ans  =

0.918

>> M(7)
ans  =

0.566
```

Matlab parcourt colonne après colonne

### 4.5 Extraction

L'opérateur : s'utilise aussi pour extraire un sous-vecteur ou une sous-matrice par énumération des indices concernés.

```
>> u
u =
0    0.2000    0.4000    0.6000    0.8000    1.0000

>> u(3:5)
ans =
0.4000    0.6000    0.8000

>> M
M  =
0.308   0.362   0.332   0.269
0.933   0.292   0.594   0.633
0.215   0.566   0.502   0.405
0.313   0.483   0.437   0.918

>> H=M(2:3,2:3) % H=M(i,j), avec 2<=i<=3, et 2<=j<=3
H  =
0.292   0.594
0.566   0.502
```

### 4.6 Suppression d'une ligne ou colonne

Grâce à la matrice vide [] on peut supprimer une ligne ou une colonne d'une matrice.

```
>> M
M  =

0.308   0.362   0.332   0.269
0.933   0.292   0.594   0.633
0.215   0.566   0.502   0.405
0.313   0.483   0.437   0.918


>> M(1,:)=[] % supprime la première ligne
M  =   ↑ parcourt la ligne

0.933   0.292   0.594   0.633
0.215   0.566   0.502   0.405
0.313   0.483   0.437   0.918


>> M(:,3)=[] % supprime la troisième colonne
M  = ↑parcourt la colonne

0.933   0.292   0.633
0.215   0.566   0.405
0.313   0.483   0.918
```

## 5 Norme d'un vecteur

Les trois normes usuelles de $\mathbb{R}^n$, à savoir :

$$\|x\|_1 = \sum_{i=1}^{n} |x_i|$$

$$\|x\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$$

$$\|x\|_\infty = \max_{i=1\ldots n} |x_i|$$

sont calculées respectivement par `norm(u,1)`, `norm(u,2)`, `norm(u,inf)`. Par défaut `norm` retourne la norme euclidienne.

```
>> x=1:4
x  =
1.   2.   3.   4.


>> n1=norm(x,1), n2=norm(x,2), n3=norm(x,inf)
n1  =
10.

n2  =
5.477

n3  =
4.
```

## 6 Norme d'une matrice

Ce paragraphe sera développé plus tard. Lorsque le premier argument de la fonction `norm` est une matrice, la fonction retourne la norme matricielle subordonnée à la norme vectorielle spécifiée dans le deuxième argument de la fonction.
Soit `x` une matrice.

- `norm(x,2)` est la plus grande valeur singulière de `x`. Les valeurs singulières de `x` sont les racines carrées des valeurs propres de $x^T x$.
- `norm(x,1)` renvoie la plus grande somme suivant les colonnes de `x`
- `norm(x,'inf')` renvoie la plus grande somme suivant les lignes de `x` la norme de Frobenius de `x` c'est à dire `sqrt(sum(diag(x'*x)))`

Dans MATLAB, le spectre d'une matrice est donné par la fonction `eig` :

```
>> A
A  =

3.   3.   3.
3.   3.   3.
```

```
3.   3.   3.

>> eig(A) % les valeurs propres sont 9 et 0 de multiplicité 2
ans  =

 -0.0000
0.0000
9.0000

>> eig(A'*A)
ans  =

-0.0000
-0.0000
81.0000

>> norm(A)
ans  =

9.0000

>> norm(A,1)
ans  =

9.

>> norm(A,inf)
ans  =

9.
```

## 7 Matrices creuses

On appelle matrice creuse, une matrice dans laquelle beaucoup de coefficients sont nuls. Il s'agit de ne stocker que les coefficients non nuls de cette matrice (et l'information nécessaire pour faire les opérations sur cette matrice). Ceci se fait par la commande `sparse`.

```
>> X=[0 0 0 1 ; 0 9 0 0 ; 0 0 0 8 ; 2 0 0 0]
X  =

0.   0.   0.   1.
0.   9.   0.   0.
0.   0.   0.   8.
2.   0.   0.   0.

>> S=sparse(X)
S =
```

```
(4,1)        2
(2,2)        9
(1,4)        1
(3,4)        8
```

L'instruction `S=sparse(X)` crée une variable de type `sparse` qui contient :

- la dimension de la matrice `( 4, 4)` et le type `sparse matrix`
- les indices (ligne, colonne) des éléments non nuls de la matrice ainsi que ces éléments.

Cette variable `S=sparse(X)` se manipule comme une matrice :

```
>> v=[4 5 6  7];

>> S*v'
ans  =

7.
45.
56.
8.
```

Lorsque la matrice creuse $X$ est trop grande pour être manipulée par MATLAB, pour représenter cette matrice, on procède comme suit :

- on construit un vecteur $u$ contenant les éléments non nuls de $X$
- on construit un vecteur $i$ d'entiers contenant les indices $i$ des éléments non nuls de $X$
- on construit un vecteur $j$ d'entiers contenant les indices $j$ des éléments non nuls de $X$
- la commande `su=sparse(i,j, u)` crée une matrice de type `sparse matrix`

```
>> u=[1 2 3 4 5 6 7 ]
u  =

1.   2.   3.   4.   5.   6.   7.

>> i=[1 2 2 3 5 6 7]
i  =

1.   2.   2.   3.   5.   6.   7.

>> j=[1 2 5 7 6 1 7]
j  =

1.   2.   5.   7.   6.   1.   7.

>> sp=sparse(i,j, u)
sp =
```

```
(1,1)        1
(6,1)        6
(2,2)        2
(2,5)        3
(5,6)        5
(3,7)        4
(7,7)        7


>> size(sp)
ans  =

7.   7.
```

La commande `full` produit une matrice pleine à partir d'une matrice creuse.
La commande `spones` produit une matrice de type `sparse matrix` dont les éléments non nuls sont égaux à 1.

```
>> full(sp)
ans  =

1.   0.   0.   0.   0.   0.   0.
0.   2.   0.   0.   3.   0.   0.
0.   0.   0.   0.   0.   0.   4.
0.   0.   0.   0.   0.   0.   0.
0.   0.   0.   0.   0.   5.   0.
6.   0.   0.   0.   0.   0.   0.
0.   0.   0.   0.   0.   0.   7.


>> spones(sp)
ans  =

(  7,  7) sparse matrix

(  1,  1)   1.
(  2,  2)   1.
(  2,  5)   1.
(  3,  7)   1.
(  5,  6)   1.
(  6,  1)   1.
(  7,  7)   1.
```

## 8 La fonction kron

La fonction MATLAB `kron` retourne le produit de Kronecker de deux matrices passées en argument.
Soit $A$ une matrice $m \times n$ et $B$ une matrice $p \times q$. Le produit tensoriel de Kronecker $C = A \otimes B$ est

défini par :

$$C = A \otimes B = \begin{pmatrix} a_{11}B & a_{12}B & \dots & a_{1n}B \\ a_{21}B & a_{22}B & \dots & a_{2n}B \\ \dots & \dots & \dots & \dots \\ a_{m1}B & a_{m2}B & \dots & a_{mn}B \end{pmatrix}$$

La matrice $C$ est de dimension $(mp) \times (nq)$.
La fonction `kron` est un outil efficace pour générer des matrices par blocs.

```
>> A=ones(3)*2

A =
2     2     2
2     2     2
2     2     2

>> K=kron(eye(3),A)

K =
2     2     2     0     0     0     0     0     0
2     2     2     0     0     0     0     0     0
2     2     2     0     0     0     0     0     0
0     0     0     2     2     2     0     0     0
0     0     0     2     2     2     0     0     0
0     0     0     2     2     2     0     0     0
0     0     0     0     0     0     2     2     2
0     0     0     0     0     0     2     2     2
0     0     0     0     0     0     2     2     2
```